GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M6: LED Board and ESP32

Company: Board&Byte **Group:** Group-8 **Date:** 7 November 2026

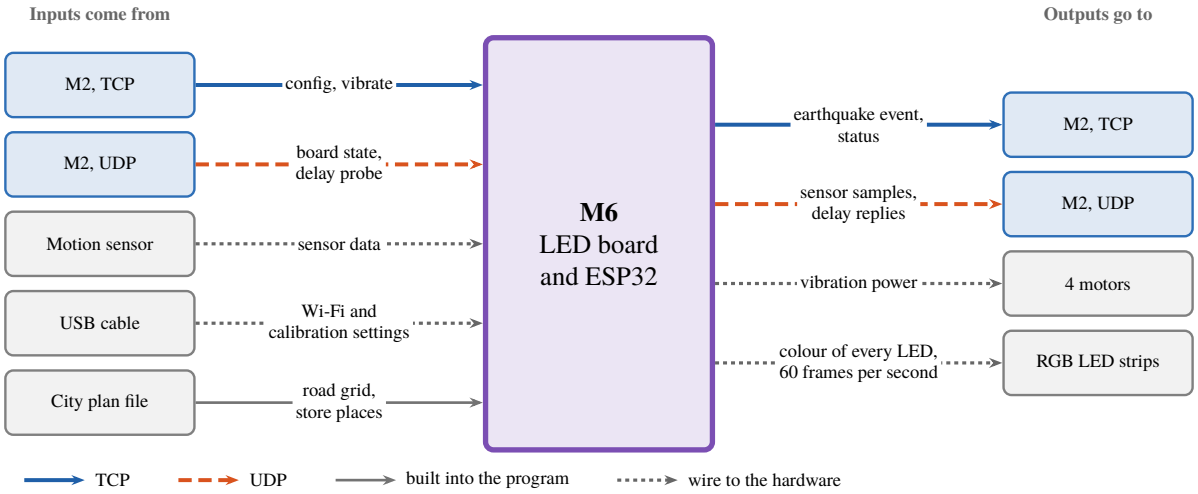
Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Hasan Alperen Elmas	1801042604
Hasan Hüseyin Özdemir	230104004048	Muhib Taha Boy	230104004165
Ayşe Poyraz	230104004085	Umutcan Ocak	220104004903

1. Purpose

M6 is the physical city board and the program on its ESP32. The board is 60 × 60 cm. Programmable addressable RGB LED strips lie in all roads and in the four stores, and there are 3D printed buildings, a motion sensor and four vibration motors. In the game the board is the only map, so it must be clear to read from every side of the table. One ESP32 runs one Arduino program with two parts. The link part gets the game state from the main computer over Wi-Fi, finds the strength and the place of a hit on the board, drives the vibration motors and reports the health of the board to the admin. The LED part turns the game state into light.

Runs on	Technology	Requirements
ESP32 in the LED board, and the board itself for the hardware	C++ with Arduino for ESP32, TCP and UDP over Wi-Fi with MessagePack messages, programmable addressable RGB LED strips with an external power supply, a motion sensor with accelerometer and gyroscope, 4 vibration motors, 3D printing, a plywood base	R2 R5 R6 R7 R8 R10 R11

M6 talks to the server over Wi-Fi and to the hardware under the board. The figure shows the data on both sides.



2. Requirements covered

M6 takes part in seven requirements. The last column says what will work when M6 is finished.

ID	Requirement	What M6 does for it	When M6 is done
R2	Delivery cycle	Delivery points blink in the player's colour, a flash when the package is delivered	Each player finds the own delivery point on the board
R5	No map on the phone	The board is the map with all motorcycles. "Find me" makes the player's dot white for 1 s	A player can always find the own motorcycle
R6	Obstacles	A red pattern for each type on the whole road segment, light blue for a puddle	Every obstacle type is clear from a distance
R7	Earthquake	Finds hand hits and motor shaking with the sensor, with a strength from 0 to 10 and a zone in a 3 × 3 grid. Drives the motors for the admin button. A shock wave from the earthquake centre	A hit on the table starts an earthquake in the game, and the earthquake is seen on the table
R8	Physical LED board	60 × 60 cm, 6 roads in each direction, LEDs on all roads, printed buildings, four stores. One ESP32 for the LEDs, the sensor and the motors brings the game state over Wi-Fi	The finished board stands on the table and shows the live game without a cable to the computer
R10	Network and measurement	Delay replies, sensor samples for the seismic graph, comes back after a lost link	The admin sees the board delay and the sensor signal live
R11	Engineering requirements	PC emulators for both sides of the link, a demo mode without a server	The server and the board can be built and tested without each other

3. Inputs and outputs

Messages that must arrive use TCP, fast and repeated data uses UDP. The board state comes 10 to 30 times per second, but the LED part draws 60 frames per second, so it fills the time between two states by itself.

Direction	Data	From / to	Content
In	Board state	M2, UDP	10 to 30 per second: phase, players (team, colour, position, flags), delivery points, obstacles, earthquake
In	Config	M2, TCP	Player colours, brightness from 0.1 to 1, power limit
In	Vibrate	M2, TCP	Motors to use, strength from 0 to 1, duration
In	Delay probe	M2, UDP	A small message with the server time, answered at once
In	Sensor	Hardware	Motion data from the accelerometer and the gyroscope
In	Settings	USB cable	Wi-Fi name and password, backup server address, calibration values
In	City plan	Shared file	Road grid and store places, turned into the LED map when the code is built
Out	Light	LED strips	The colour of every LED, 60 frames per second
Out	Vibration	4 motors	Motor power with a soft start and safety limits
Out	Earthquake event	M2, TCP, to M1 and M5	Source (hand or motors), strength from 0 to 10, centre, zone from 0 to 8, confidence
Out	Sensor samples	M2, UDP, to M5	Sensor data for the seismic graph, sent in small groups
Out	Status, delay reply	M2	Status on TCP once per second: frame rate, Wi-Fi signal, power estimate, errors. Delay replies on UDP

4. Main components

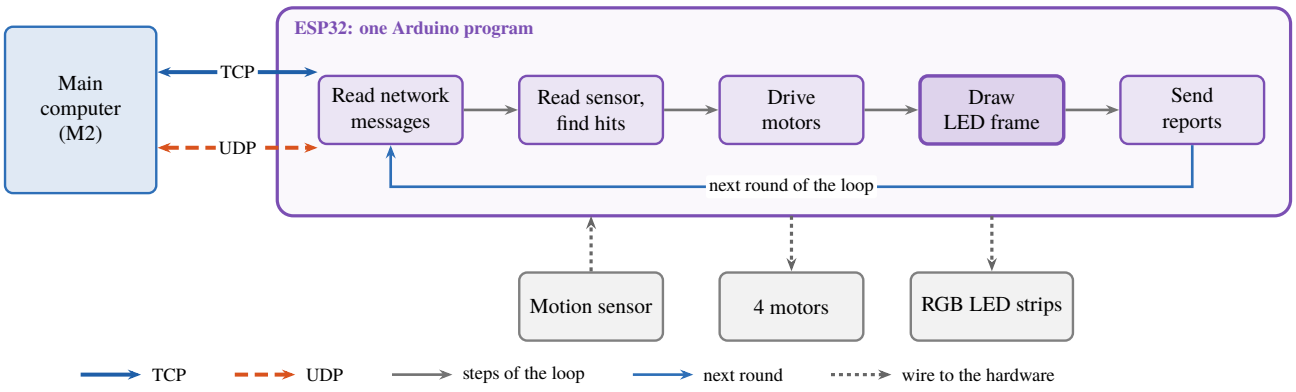
M6 has eleven parts. The first six are the link part and the last five are the LED part. All of them run in the main loop of the Arduino program.

Component	Takes	Gives
Network link	Wi-Fi, server address	One TCP channel and one UDP stream, a new link after a loss
Message codec	Raw messages	Checked messages. Old, broken or wrong version messages are dropped
State hand over	Checked board state and config	The newest full state for the LED part
Hit detector	Sensor samples	Earthquake events with strength, zone and confidence
Motor controller	Vibrate commands	Motor power with a soft start and safety limits
Status and settings	Frame report, USB settings	Status reports, delay replies, settings saved in flash
LED map	City plan	The place of every LED on the roads and in the stores, made by a script
Motion smoother	The last two board states	Smooth motorcycle positions along the roads for every frame
Layers and effects	Board state, phase, link state	Five layers and the patterns for obstacles, earthquakes and game phases
Post processing and output	The mixed frame, config	Corrected colours inside the brightness and power limit, sent to the strips
Demo mode	A built in scenario	A full game look without a server

5. How it works

One program with a main loop:

The ESP32 runs one Arduino program. The main loop repeats the same steps again and again: it reads the new network messages, reads the sensor, drives the motors, lets the LED part draw one LED frame and sends the reports. The link part keeps the newest full board state in memory. At the start of each frame the LED part reads this state, so it always draws the latest game.

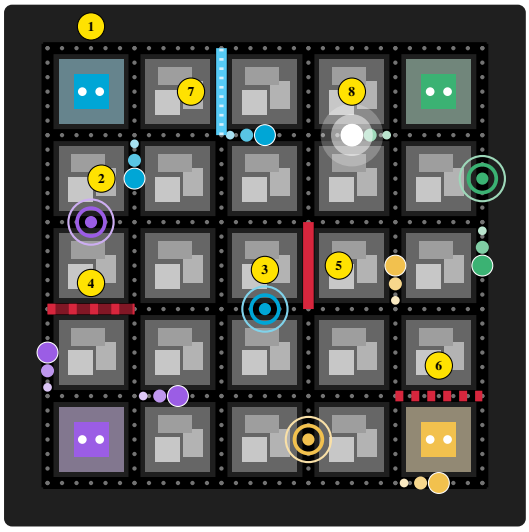


The board:

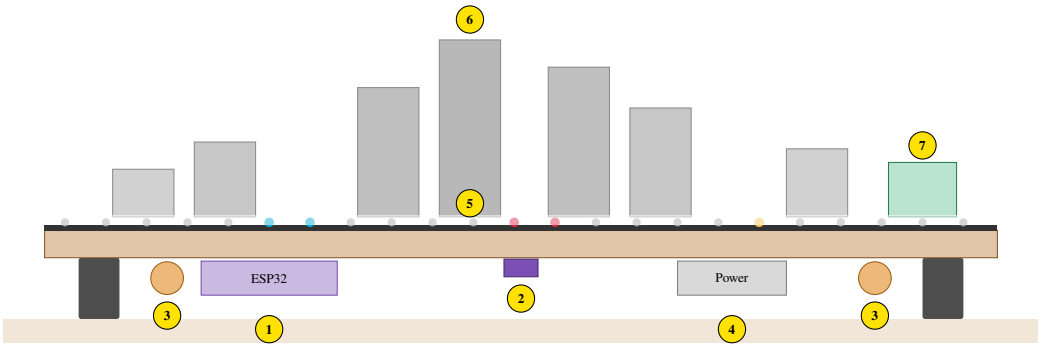
The base is plywood. On it there are 6 roads in each direction, so the city has 5 × 5 blocks. The LED strips lie in the roads between the crossings. There are no LEDs at the crossings, because two strips cannot lie on top of each other. The buildings are 3D printed from the same models as the 3D city on the phone, so a player finds the same building on the board and on the phone. The LEDs need an external power supply, and the software keeps the power under the limit in the config.

What is inside the board:

The figure below is a side cut of the board. The motion sensor is fixed at the centre, under the plywood, so it feels a hit from every side. The four vibration motors are under the four corners. The ESP32 and a power supply for the LEDs and the motors sit under the board too.



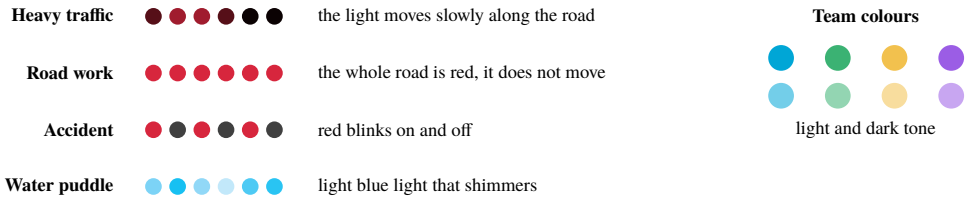
1 Team store in the team colour
2 Motorcycle: a bright dot with a short tail
3 Delivery point: blinks in the player's tone
4 Heavy traffic: a slow dim red wave
5 Road work: bright steady red
6 Accident: red blinking
7 Water puddle: light blue shimmer
8 "Find me": white flash for 1 second



1 ESP32
2 Motion sensor at the centre
3 Vibration motors at the corners
4 Power supply
5 RGB LED strips on the roads
6 3D printed buildings
7 Team store with LEDs

Colours:

There are four teams and each team has one colour: blue, green, yellow and purple. The two players of a team have a light and a dark tone of the team colour. Red is used only for obstacles, so red always means "this road slows you down". The pattern of the red tells the type of the obstacle. A water puddle is light blue, because it makes the road slippery, not slow. White is used only for "Find me", so a player always knows that the white dot is the own motorcycle.

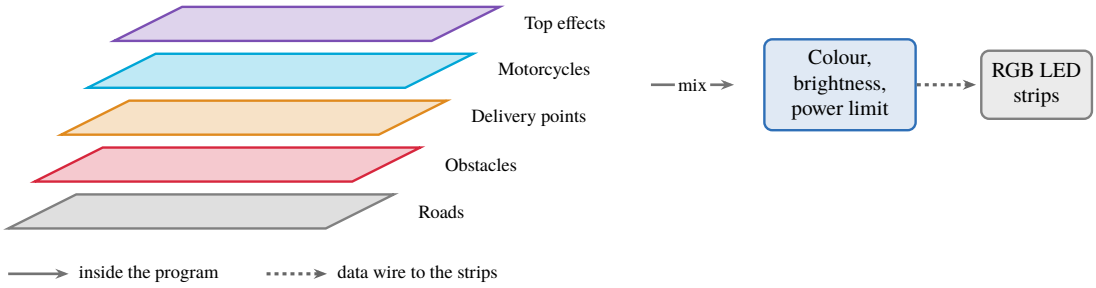


Layers:

The LED part draws 60 frames per second. Each frame is made from five layers, from the bottom to the top: the roads, the obstacles, the delivery points, the motorcycles and a top layer for short effects. A higher layer covers a lower one, so when two things fall on the same LED, the higher one is shown. When two motorcycles are on the same LED, their colours take turns. After the layers, the LED part corrects the colours for the eye, sets the brightness from the config, and lowers all LEDs a little if the frame would need more power than the limit.

Smooth motion on the roads:

The board state comes less often than the frames, so the motion smoother moves each motorcycle dot between the last two states. It moves the dot along the roads, through the crossing, and never in a straight line over a block. The dot is drawn one state behind the newest one, so it adds a small delay. If a state is lost, the dot keeps moving along its road with its last speed until the next state comes. The dot has a short tail, so the direction of motion is easy to see.



Two channels:

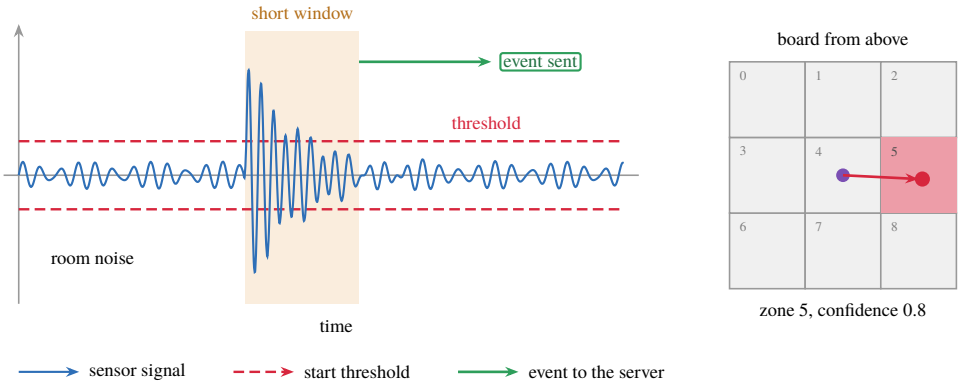
The board keeps one TCP channel and one UDP stream to the server. Messages that must arrive go on TCP: the config, the vibrate commands, the earthquake events and the status report. Each TCP message starts with its length, because TCP is only a stream of bytes. Data that comes again and again goes on UDP: the board state, the sensor samples and the delay probes. If one UDP packet is lost, the next one replaces it, so the LEDs never wait. Every message has an order number, so the board drops a UDP packet that comes late. Each start of the server makes a new session number. When the board sees a new session number, it starts its order numbers again, so it does not drop new messages as old ones.

Finding a hit:

The sensor measures the motion all the time. First a filter takes away gravity and slow tilting, so only the shaking is left. The detector also learns the normal noise of the room and puts the start threshold well above it. So the threshold goes up on a noisy table, and normal play does not start an earthquake. When the signal passes the threshold, a hit starts, and the detector takes a short window from this moment. The stronger the shaking in this window, the higher the strength. Like a real earthquake scale, the strength grows slowly for very strong hits, so a light tap and a hard hit both get a useful value. The scale is set once in calibration with a standard hit, and the result is kept between 0 and 10.

Finding the zone:

The accelerometer shows the direction of the first push of the board, and the gyroscope shows how the board tilts. Together they show the side of the hit, and the size of the push shows how far the hit is from the centre. From this the detector picks one of 9 zones in a 3 × 3 grid and gives a confidence value. After a hit the detector waits a short time, so one hit makes only one event.

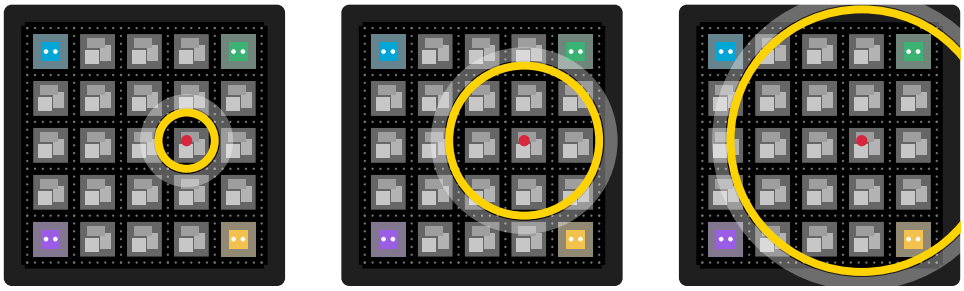


Motors:

The four motors are under the corners, at zones 0, 2, 6 and 8. When the admin presses the earthquake button, M1 sends a vibrate command on TCP. The command says which motors to use, the strength and the time. When the admin chooses one zone, M1 uses the motors nearest to it. For the whole board it uses all four. M6 starts the motors slowly, so the power does not jump. While the motors run, the detector stops learning the room noise, and the whole shaking gives only one event with "motors" as its source. So the earthquake in the game follows the real shaking of the table, and the motors do not start a second earthquake.

Earthquake on the board:

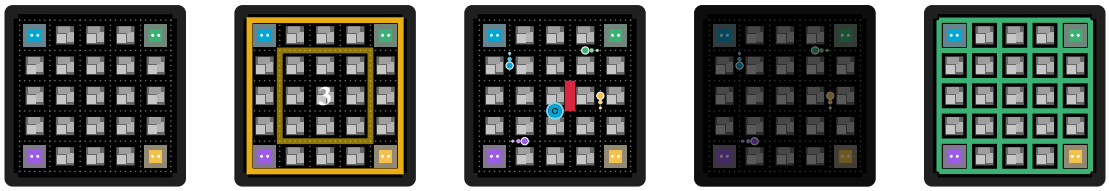
When an earthquake starts, a ring of light grows from the centre of the earthquake over the whole board, like a wave on water, and slowly fades. The road work after an earthquake is not a decision of M6. M1 puts it into the obstacle list, and M6 only draws it like any other obstacle until M1 removes it from the list.



start a moment later later

Game phases and the link:

Every game phase has its own look, so the people around the table always know what happens.



Lobby stores glow, roads dark

Countdown light moves in, 3, 2, 1

Running the live game

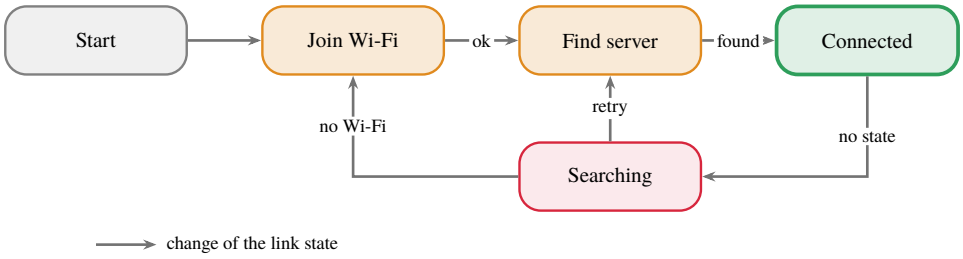
Paused all dims and breathes slowly

Finished roads in the winner colour

In the lobby the roads are dark and the four stores glow in the team colours. In the countdown the light moves from the edge to the centre in three steps, one for each number. During the game the board shows the live state. In a pause everything stays where it is, but all LEDs dim and breathe slowly. At the end all roads turn to the colour of the winner team, and in a draw the colours of the winners take turns.

When the link is lost:

If no state comes for a short time, M6 stops the motors at once and the link state becomes “searching”. The roads go dark and the stores breathe slowly in a dim white, so everybody can see that the board is waiting. Then M6 tries to connect again: first to the server it finds on the local network, then to the backup address. The Wi-Fi name, the password, the backup address and the calibration values are stored in the ESP32’s flash memory. We can change them over the USB cable, so we do not need to build the code again at a new place.



6. Testing

M6 is tested without the server in two ways. For the link part two PC programs are used. A server emulator takes the place of M2. It sends test board states, config and vibrate commands with delay and loss that we can change, and it shows the earthquake events and the seismic signal. A board emulator takes the place of the real board, so the server side can also be tested without it. We check the shared example messages, broken packets, a long run, heavy loss, a router restart and a server restart. For the sensor we hit each of the 9 zones many times and check that hits are found, that the zone is right or a neighbour zone, and that the motors do not start a false earthquake. The LED part is tested on the real board with a built in demo scenario. It gives recorded board states as test data and plays a full game with every animation and every game phase. Test patterns check the order and place of the LEDs, the 8 player colours (three people must name each colour correctly), the power limit at full white and the heat in a long run.